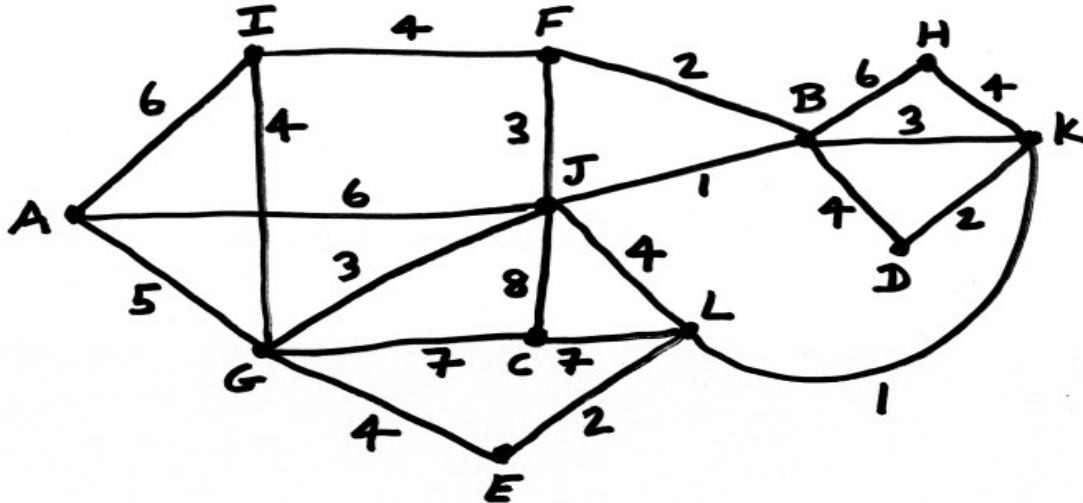


INT202 Complexity of Algorithms

Consider the weighted (undirected) graph below. (Edges $\{G, I\}$ and $\{A, J\}$ cross but there is not a vertex at their intersection.)



Applying Dijkstra's algorithm to find shortest paths from vertex A to all other vertices in the graph. You should list the shortest distance array and draw the subgraph that contains only those edges used in the shortest paths.

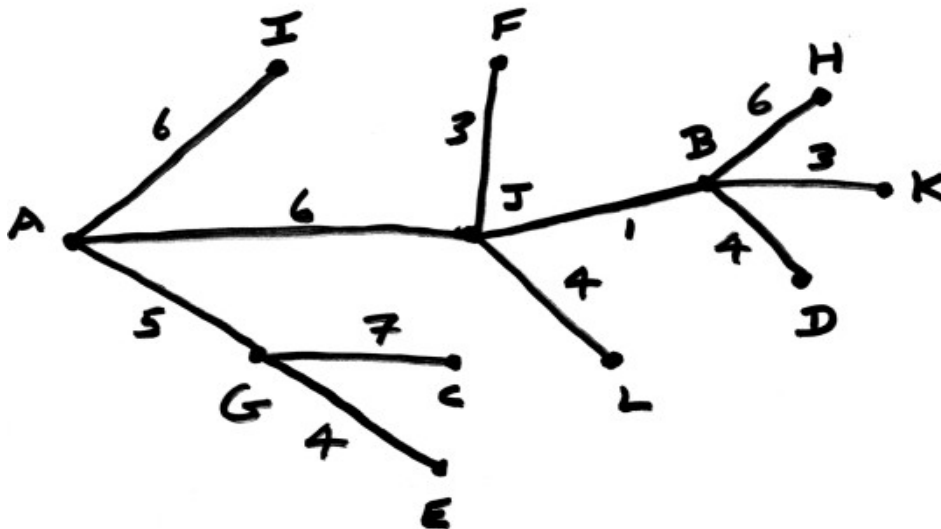
Applying Dijkstra's algorithm, we can construct the following table wherein we keep track of the current minimum distances, as well as the predecessor on the path to that vertex. Note that the predecessor of A is listed as $\emptyset$ since the vertex A has no predecessor. For lack of room, we don't list all of the vertices that are currently fixed in the first column, just the new vertex that becomes fixed in that round. In this case, after the vertex B becomes fixed, it turns out that none of the distances decrease in the following steps. In general, it might take longer for other distances to decrease to their proper values.

Fixed/C	D&P											
	A	B	C	D	E	F	G	H	I	J	K	L
-	0,null	∞ ,null	∞ ,null	∞ ,null	∞ ,null	∞ ,null	∞ ,null	∞ ,null	∞ ,null	∞ ,null	∞ ,null	∞ ,null
A	0,null	∞ ,null	∞ ,null	∞ ,null	∞ ,null	∞ ,null	5,A	∞ ,null	6,A	6,A	∞ ,null	∞ ,null
G	0,null	∞ ,null	12,G	∞ ,null	9,G	∞ ,null	5,A	∞ ,null	6,A	6,A	∞ ,null	∞ ,null
I	0,null	∞ ,null	12,G	∞ ,null	9,G	10,I	5,A	∞ ,null	6,A	6,A	∞ ,null	∞ ,null
J	0,null	7,J	12,G	∞ ,null	9,G	9,J	5,A	∞ ,null	6,A	6,A	∞ ,null	∞ ,null
B	0,null	7,J	12,G	11,B	9,G	9,J	5,A	13,B	6,A	6,A	10,B	10,J
E	0,null	7,J	12,G	11,B	9,G	9,J	5,A	13,B	6,A	6,A	10,B	10,J
F	0,null	7,J	12,G	11,B	9,G	9,J	5,A	13,B	6,A	6,A	10,B	10,J
K	0,null	7,J	12,G	11,B	9,G	9,J	5,A	13,B	6,A	6,A	10,B	10,J
L	0,null	7,J	12,G	11,B	9,G	9,J	5,A	13,B	6,A	6,A	10,B	10,J
D	0,null	7,J	12,G	11,B	9,G	9,J	5,A	13,B	6,A	6,A	10,B	10,J
C	0,null	7,J	12,G	11,B	9,G	9,J	5,A	13,B	6,A	6,A	10,B	10,J

H	0,null	7,J	12,G	11,B	9,G	9,J	5,A	13,B	6,A	6,A	10,B	10,J
---	--------	-----	------	------	-----	-----	-----	------	-----	-----	------	------

The predecessors allow us to determine the path from vertex A to any other vertex by tracing backwards through the path. For example, to get to vertex K, we come through vertex B, which is reached via vertex J, and we get to J from A. Thus the path from A to K is A, J, B, K. In a similar manner we can determine any of the other shortest paths from vertex A to other vertices.

Finally, we draw the subgraph that contains only those edges used in the shortest paths.



Note that the subgraph generated in this manner is necessarily a spanning tree of the graph, since each vertex (except the starting vertex) has a unique predecessor. This will not, in general be a minimum spanning tree of the original graph. Also, as in the case of finding minimum spanning trees, there may be more than one subgraph that could be generated by applying Dijkstra's algorithm. The differences that could arise will depend upon how ties are broken during selecting which vertex becomes fixed during the next round of the algorithm.